Competitive Programming

Section 43: String Algorithms I

Competitive Programming Course

2026-10-19

Table of contents

1 What this section adds **3**

2 Prefix-function thinking **4**
2.1 Borders: one block at both ends 4
2.2 The prefix function 4
2.3 The update rule 5

3 KMP matching **7**

4 Borders and periods **8**
4.1 Every border, largest first 8
4.2 The shortest period 8

5 Implementation, linearity, and debugging **10**
5.1 The code 10
5.2 Why it is linear 12
5.3 Language notes 12
5.4 Tests before trusting a solution 12

6 Exercises **14**

1 What this section adds

Until now, strings have been arrays of characters: we sorted them, counted their letters, and compared them in loops. This section starts treating a string as an object with *structure*, and the first structural question is the most common one in practice: **where does a pattern occur in a text?**

The obvious method tries every starting position and compares character by character. On ordinary text it stops early and looks fast, but on repetitive input it does not:

```
text    = aaaaaaaaaaaaaaaaaaaaaaaaaaaaab      (n characters)
pattern = aaaaaaaab                          (m characters)
```

Every alignment matches `m-1` characters before failing on the `b`, so the total work is about `(n - m) * m`. With `n = 2*10^5` and `m = 10^5` that is about `10^10` comparisons. The waste is that each failed attempt forgets what it just read: after matching `aaaaaaaa` we *know* the next alignment starts with seven matching `a`'s, yet the naive loop compares them all again.

The cure is to precompute, once, how the pattern overlaps with itself. If a partial match of length `j` fails, the longest prefix of the pattern that is also a suffix of those `j` characters tells us exactly where to resume, with no step backwards in the text. That overlap length is the **prefix function**. It is computed in linear time, and the pattern search built on it, the Knuth–Morris–Pratt algorithm, runs in `O(n + m)`.

The same array answers more than matching, so the section is organized around four questions about one string:

Question	Problem	Tool
Where does a pattern occur in a text?	A	KMP on `pattern + '#' + text`
Which prefixes are also suffixes (all of them)?	B	the chain `pi[n-1]`, `pi[pi[n-1]-1]`, ...
What is the shortest repeating block?	C	`n - pi[n-1]`, if it divides `n`
How often does each prefix occur?	D	counting along failure links

Nothing here needs earlier string machinery: only arrays, loops, and an amortized-counting argument like the one used for two pointers in Section 6. Section 44 then adds tries, palindrome queries, and suffix automata, and Section 55 adds hashing; the prefix function remains the simplest exact matching tool of them all.

2 Prefix-function thinking

2.1 Borders: one block at both ends

Look at the string `abacaba`. It starts with `aba` and it also ends with `aba`: the same block appears at both ends. A string that is at the same time a prefix and a suffix of `s` is called a **border** of `s`.

It helps to picture a border as **two copies of one block**:

- the **prefix copy** sits at the left end, starting at position `0`;
- the **suffix copy** sits at the right end, finishing at the last position.

```
s       = a b a c a b a
prefix  = a b a . . . .       positions 0..2
suffix  = . . . . a b a       positions 4..6
```

The borders of `abacaba` are `a` and `aba`. The whole string is also a prefix and a suffix of itself, but that is true of every string and tells us nothing, so we only keep **proper** borders, those shorter than `s`. The two copies are allowed to overlap each other: in `aaaa` the border `aaa` has its prefix copy at positions `0..2` and its suffix copy at positions `1..3`.

Why borders matter is exactly the question left open in the introduction. Suppose we have just matched the characters `abacaba` of a pattern and the next character fails. The suffix copy `aba` is the end of what we just read, and the prefix copy says that those same three characters are also the *beginning* of the pattern. So we may continue as if `aba` were already matched, without moving backwards in the text. The longer the border, the more work we keep, which is why we always want the **longest** proper border.

2.2 The prefix function

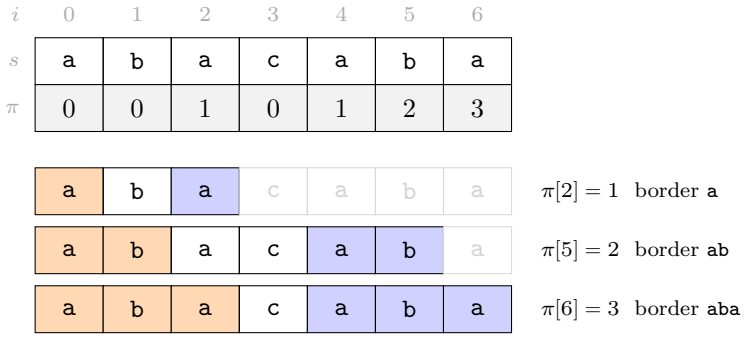
A failure can happen after any number of matched characters, so we need the longest border of every prefix of the string, not only of the whole string. That is the prefix function:

```
pi[i] = length of the longest proper border of s[0..i]
      = length of the longest proper prefix of s[0..i]
        that is also a suffix of s[0..i]
```

Proper means the border is shorter than `s[0..i]`, so `pi[i] < i+1` always, and `pi[i] = 0` when no border exists. Storing a length is enough: the two copies are equal by definition, so knowing where the prefix copy ends tells us everything, and we never copy the substring itself.

In the figure below, the top rows give the positions, the letters of `abacaba`, and the value of `pi` under each letter. Each lower row looks at one prefix `s[0..i]`: the orange cells are the prefix copy, the blue cells are the suffix copy, and the faded cells lie beyond position `i`. For `i = 6` the

copies are `aba` at both ends, so `pi[6] = 3`. The same string `abacaba` is the running example for the whole section.



Prefix copy (starts at position 0) and **suffix copy** (ends at position i) hold the same letters. The border is *proper*: each copy is shorter than $s[0..i]$, although the two copies may overlap each other.
Faded cells lie beyond position i and are not part of the prefix being measured.
π stores only the *length* of the copies.

2.3 The update rule

Positions are filled left to right, so when we reach `s[i]` every earlier value `pi[0..i-1]` is known. The question is how to reuse them.

Key observation. Take any border of `s[0..i]` of length `k > 0`. Remove its last character from both copies: what remains is a border of `s[0..i-1]` of length `k-1`, and the removed character satisfies `s[k-1] = s[i]`. Read backwards, this says:

> every border of `s[0..i]` is a border of `s[0..i-1]` extended by one character, namely `s[i]`.

So the candidates for `pi[i]` are the borders of the previous prefix, and for each candidate of length `j` there is a single test: is `s[j] == s[i]`? (`s[j]` is the character just after the prefix copy, the one the extension needs.) We want the longest border, so we try the candidates from longest to shortest.

The longest candidate is `pi[i-1]`. The next ones are the borders of that border: `pi[pi[i-1]-1]`, and so on, because a border of a border is again a border (the figure below shows the chain). That gives the whole rule:

```
j = pi[i-1]                       # longest border of s[0..i-1]
while j > 0 and s[i] != s[j]:     # it cannot be extended by s[i]:
    j = pi[j-1]                   #   try the next shorter border
if s[i] == s[j]:                  # extend the surviving border
    j += 1
pi[i] = j
```

A complete trace. Extend the running example by one letter, to `abacabab`, so that every case of the rule appears:

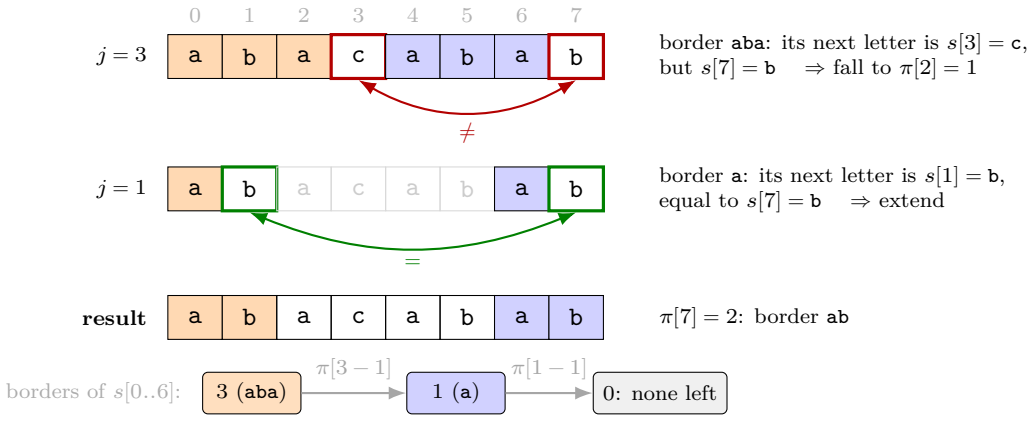
i	s[i]	start j = pi[i-1]	what happens	pi[i]
1	b	0	s[0] = a differs, nothing to fall back to	0
2	a	0	s[0] = a matches: extend	1
3	c	1	s[1] = b differs, fall to pi[0] = 0; s[0] = a differs	0
4	a	0	s[0] = a matches: extend	1
5	b	1	s[1] = b matches: extend	2
6	a	2	s[2] = a matches: extend	3
7	b	3	s[3] = c differs, fall to pi[2] = 1; s[1] = b matches: extend	2

Rows 2, 4, 5 and 6 are the easy case: the previous border extends at once. Row 3 falls all the way to 0 and still fails. Row 7 is the interesting one:

```
s[0..6]          = a b a c a b a     longest border  aba   (j = 3)
extend by b?       s[3] = c  !=  b   fails
next border      = pi[2] = 1         border  a             (j = 1)
extend by b?       s[1] = b  ==  b   succeeds
s[0..7]          = a b a c a b a b   longest border  ab    pi[7] = 2
```

Two things in row 7 deserve attention. We never re-read the text: the fallback jumped straight from 3 to 1 using a stored value, without trying length 2. And the jump is safe: a border of length 2 of `s[0..6]` would have to be `ab` at both ends, but `s[0..6]` ends in `ba`, and the chain `3 -> 1 -> 0` lists exactly the borders that exist, so nothing is skipped.

The `while` loop is therefore not a search. If the candidate of length `j` is contradicted, every longer candidate was contradicted before it, so the longest one still alive is the longest border *of that border*, `pi[j-1]`, a value already computed.



Orange is the prefix copy of the candidate border, blue its suffix copy. The only letter ever tested is the one just after the prefix copy, $s[j]$, against the new letter $s[i]$. When it fails, the next candidate is the next link of the chain, read from π: the text is never re-read.

3 KMP matching

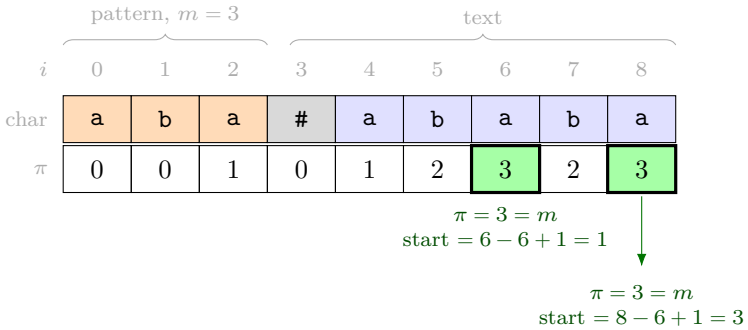
To find every occurrence of a pattern `p` of length `m` in a text `t`, run the prefix function once over

```
p + separator + t
```

The separator must be a character that cannot appear in either string. Its only job is to stop a prefix from stretching across the join, which would report matches that do not exist. If the alphabet is not under your control, do not concatenate at all; run the matching loop directly against `p` and `t` with the same fallback rule.

Wherever `pi[i] == m`, the pattern has just finished. Counting back over the pattern, the separator, and the matched copy gives the 1-based start in the text:

```
start = i - 2*m + 1
```



Why overlapping matches are not lost. At $i = 7$, just after a full match, the candidate is $j = 3$. The next character `b` contradicts it, so the loop falls to $\pi[2] = 1$ — the longest border of the pattern — and `b` matches there, giving $\pi[7] = 2$. One more character reaches 3 again at $i = 8$. A naive scanner that restarted at 0 after a match would skip the occurrence at position 3.

Why the text index never retreats. Only j moves backwards, and it never falls below 0 while i only advances. Each character raises j by at most one, so total fallback is bounded by total increase: $O(n)$.

Overlapping occurrences come out for free, and that is the whole advantage over naive scanning: after a match the algorithm falls back to the longest border of the pattern, never to zero, so characters already known to match are never compared again.

4 Borders and periods

4.1 Every border, largest first

The border chain from Section 2.3 is also the answer to problem B. The longest border is `pi[n-1]`; the next is the longest border of *that* prefix, and so on until the chain reaches zero:

```
k = pi[n-1]
while k > 0:
    record k
    k = pi[k-1]
```

This produces borders in decreasing order. Reverse the list if the statement asks for increasing order. For `abacaba` it yields `3, 1`; for `abcabca` it yields `4, 1`; for `abc` it yields nothing at all.

4.2 The shortest period

The question. Problem C asks for the shortest block that, repeated a whole number of times, gives the string. `abcabcabc` is three copies of `abc`, so the answer is `3`; `abacaba` is not made of copies of anything shorter, so the answer is `7`. The prefix function answers this in two steps: borders give a lower bound, and a divisibility test decides whether the bound is reached.

Step 1: shifts and a lower bound. Write the string twice, the second copy moved `p` places to the right. The shift is *consistent* when the overlapping letters agree, that is when `s[i] = s[i+p]` for every `i` where both exist. For `ababab` and `p = 2`:

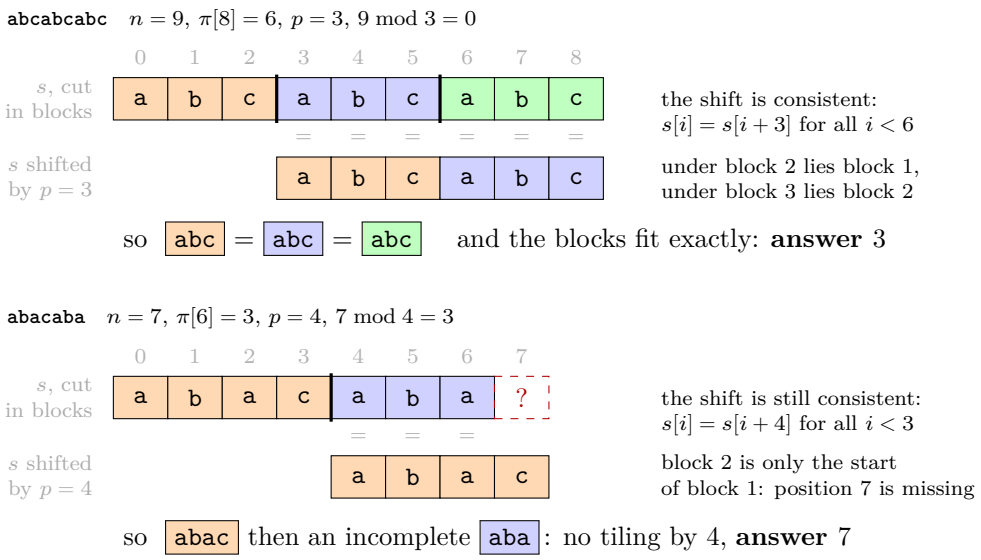
```
a b [a b a b]
    [a b a b] a b      the 4 bracketed letters agree
```

In the top copy the bracketed overlap is the end of the string; in the bottom copy it is the start. So the shift is consistent exactly when the end and the start agree, which is the definition of a border: every border of length `b` gives a consistent shift `p = n - b`, and the longest border gives the smallest one,

```
p = n - pi[n-1].
```

A block of length `q` that tiles the string also gives a consistent shift (shifting by one block lines up equal blocks), so `q >= p`. No block is shorter than `p`.

Step 2: when does `p` itself tile the string? Tiling by blocks of length `p` needs two things: the blocks must all be equal, and they must fit exactly. The first is already guaranteed by the consistent shift. Cut the string into blocks of length `p`: letter `r` of block `t+1` is `s[tp + r]`, which equals `s[(t-1)p + r]`, letter `r` of block `t`. Each block equals the previous one, so all blocks equal the first. Only the **last** block can go wrong, by being shorter than `p`. The figure shows both situations.



Whether the last block is complete is exactly the question `n % p == 0`. That is why, once `p` is known, the modulo alone decides.

Step 3: when the test fails, the answer is `n`. Could some longer block `q`, with `p < q < n` and `n % q == 0`, still tile the string? No. Such a `q` is also a consistent shift, and `q <= n/2` because at least two blocks fit, so `p + q <= n`. A classical lemma on periods (Fine and Wilf) then says that `gcd(p, q)` is a consistent shift too. It cannot be smaller than `p`, the smallest one, so `p` divides `q`, and therefore `p` divides `n`: the test would not have failed.

The algorithm.

```
b = pi[n-1]
p = n - b
answer = p if n % p == 0 else n
```

String	`n`	`pi[n-1]`	`p`	`n % p`	Answer
`abcabcabc`	9	6	3	0	3
`ababab`	6	4	2	0	2
`abacaba`	7	3	4	3	7
`abcdxyabcd`	10	4	6	4	10
`ababa`	5	3	2	1	5

`ababa` is the smallest trap worth memorizing: the shift by `2` is consistent, but `2` does not divide `5`, so the answer is `5`.

5 Implementation, linearity, and debugging

5.1 The code

Everything in this section rests on one function. The other three are a few lines each on top of it: KMP runs it over `p + '#' + t`, the border chain follows `pi` from the last position, and the period test reads only `pi[n-1]`. These are the functions that produced every number in the figures.

```
// pi[i] = length of the longest proper border of s[0..i]
vector<int> prefix_function(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];                    // best border of s[0..i-1]
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];                    // fall back to a shorter border
        if (s[i] == s[j]) j++;                // extend it by s[i]
        pi[i] = j;
    }
    return pi;
}

// 1-based starts of every occurrence of p in t (overlaps included)
vector<int> kmp_occurrences(const string& p, const string& t) {
    int m = p.size();
    string s;
    s.reserve(m + 1 + t.size());
    s += p;
    s += '#';                                 // appears in neither string
    s += t;
    vector<int> pi = prefix_function(s), starts;
    for (int i = 2 * m; i < (int)s.size(); i++)
        if (pi[i] == m) starts.push_back(i - 2 * m + 1);
    return starts;
}

// all proper borders, largest first (problem B prints them reversed)
vector<int> border_chain(const string& s) {
    vector<int> pi = prefix_function(s), out;
    for (int k = pi.back(); k > 0; k = pi[k - 1]) out.push_back(k);
    return out;
}

// shortest period that tiles the whole string (problem C)
```

```cpp
int minimal_period(const string& s) {
    int n = s.size(), p = n - prefix_function(s).back();
    return n % p == 0 ? p : n;
}
```

```python
def prefix_function(s):
    """pi[i] = length of the longest proper border of s[0..i]."""
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        j = pi[i - 1]                       # best border of s[0..i-1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]                   # fall back to a shorter border
        if s[i] == s[j]:
            j += 1                          # extend it by s[i]
        pi[i] = j
    return pi


def kmp_occurrences(p, t):
    """1-based starts of every occurrence of p in t (overlaps included)."""
    m = len(p)
    pi = prefix_function(p + b"#" + t)      # '#' appears in neither string
    return [i - 2 * m + 1 for i in range(2 * m, len(pi)) if pi[i] == m]


def border_chain(s):
    """All proper borders, largest first (problem B prints them reversed)."""
    pi = prefix_function(s)
    out = []
    k = pi[-1]
    while k > 0:
        out.append(k)
        k = pi[k - 1]
    return out


def minimal_period(s):
    """Shortest period that tiles the whole string (problem C)."""
    n = len(s)
    p = n - prefix_function(s)[-1]
    return p if n % p == 0 else n
```

The Python version works on `bytes` (read with `sys.stdin.buffer.read()`), so `p + b"#" + t` builds the combined string once and `s[i] != s[j]` compares small integers. The `while` loop in `prefix_function` is the fallback of the update rule; the `if` after it is the one-character extension. In `kmp_occurrences` the scan starts at index `2*m`, the first position where a full copy of the pattern can end inside the text.

On the running examples the four functions return:

Call	Result	Why
`prefix_function("abacaba")`	`0 0 1 0 1 2 3`	the table of Section 2.2
`kmp_occurrences("aba", "abababa")`	`1 3 5`	overlaps are all found
`border_chain("abcabca")`	`4 1`	borders `abca`, then `a`
`minimal_period("abacaba")`	`7`	`7 - 3 = 4` does not divide `7`

5.2 Why it is linear

The index `i` never moves backwards. The pointer `j` rises by at most one per position, so across the whole run it rises at most `n` times. Every iteration of the `while` loop strictly lowers `j`, and `j` never drops below zero, so the total number of fallback steps is bounded by the total number of increases. Both are `O(n)`, which is why a loop that looks quadratic is not.

The loop invariant behind that argument: before position `i` is processed, `j` is the longest border of `s[0..i-1]` not yet contradicted by the new character. Each fallback preserves the longest surviving candidate; each success extends it by exactly one.

5.3 Language notes

C++. A `vector<int>` for `pi` is enough. Reserve the combined string once rather than appending in a loop. Occurrence counts can exceed `int` on adversarial inputs such as `aaaa...a`, so accumulate in `long long`.

Python. Read with `sys.stdin.buffer.read()` and work on `bytes`; indexing a `bytes` object yields integers and compares faster than slicing strings. Never build the combined string by repeated `+=` in a loop.

5.4 Tests before trusting a solution

Case	What it catches
pattern longer than text	negative or out-of-range start positions
pattern equals text	off-by-one in `i - 2*m + 1`
`aba` in `ababa`	a scanner that restarts at zero and loses overlaps
`aaaa` in `aaaaa`	the maximal-overlap stress case
no occurrence at all	an empty-output path
`ababa` for the period	the missing `n % p == 0` divisibility test
`abc` for borders	an empty border chain
separator present in input	a bogus match across the join

Useful values to check by hand: `pi("aaaaa") = 0 1 2 3 4`, the proper borders of `abcabca` are `1` and `4`, and `abacaba` has candidate period `4` but answer `7`.

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/43_string_algorithms_i/check.py
```

6 Exercises

Use A to get the combined-string indexing and the 1-based start formula right, B to walk the border chain to exhaustion rather than stopping at the first link, and C to apply the divisibility test that turns a candidate period into a real one. Use D as the blind synthesis problem: every prefix needs a count, not just the whole string, so decide what a failure link contributes to its target before writing any code. Test a pattern longer than the text, a pattern equal to the text, overlapping matches, an all-same-character input, and a string with no border at all.

- A. KMP Occurrences: Print the number and 1-based positions of all occurrences of pattern in text.
- B. All Borders: Print all proper border lengths of a string in increasing order.
- C. Minimal Period: Print the shortest period length if the string is repetitions of a block, otherwise `n`.
- D. Prefix Frequency: For every prefix length, print how many times that prefix occurs as a substring.